

프로그래머를 위한 범주론2

-정지문제-

김학재

2026.07.21

목차

1. 지난시간
2. 정지문제
3. 칸토어의 대각선 논법
4. 마무리

지난시간

범주론은 어떻게 연결되는가만 본다는 것이다.

객체는 추상적이고 모호하다라고 보며 객체 내부는 관심이 없다.

객체를 합성하기 위해 내부구현을 알아야한다면 객체 지향의 장점을 잃는것이다.

범주 정의

1. 합성할 수 있어야하고,

$$A \xrightarrow{f} B \xrightarrow{g} C$$

- >

$$A \xrightarrow{h} C$$

2. 결합성 만족

$$h \circ (g \circ f) = (h \circ g) \circ f = h \circ g \circ f$$

3. 모든객체에는 항등사상 존재

$$A \xrightarrow{id} A$$

정지문제

수학에서 함수는 항상 같은 값을 돌려준다.

$$f(x) = x + 1$$

$$\text{Int} = \{ \dots, -2, -1, 0, 1, 2, \dots \}$$

하지만 Haskell 에서는 함수가 항상 결과를 내는 것은 아니다.(다른 프로그래밍 언어도 마찬가지)

1. `loop x = loop x`

```
2. int loop(int x) {  
    return loop(x);  
}
```

-> 재귀상황 발생

$\perp$ (bottom) : 프로그램이 정상적인 값을 반환하지 못하고 무한루프 및 에러인 상태 의미

$$\text{Int} = \{ \perp, \dots, -2, -1, 0, 1, 2, \dots \}$$

여기서 정지문제가 등장한다.

함수가 $\perp$ 를 반환할지, 아니면 정상적인 값을 반환할지 미리 알 수 없나?

정지문제

Halting Problem (정지문제) : 앨런 튜링 1936년에 논리학적으로 결정불가능함을 증명함.

주어진 프로그램이 해결하고자 하는 문제가 해결가능한지의 여부를 말해줄 수 있는 일반화된 알고리즘이 존재하는가?

-> 어떤 프로그램이 어떤 입력값을 받았을 때 정상적인 값을 계산할지, 아니면 계산이 완료되지 못한 상태가 될 지 프로그램을 돌려보기 전에 알 수 있는가.

컴퓨터로 풀기 쉬운 문제 : 여러가지 알고리즘들을 활용한 문제들.

컴퓨터로 해결 불가능한 문제 :

- 앨런튜링이 정지문제 컴퓨터로 해결불가능함 증명. (역사상 첫번째)
- 문맥 자유 문법의 동등성 문제 (두 개의 프로그래밍 언어 규칙이 완전히 똑같은지 판별하기)
- 바쁜 비버 문제 (제한된 상태를 가진 머신이 최대 출력할 수 있는 글자 수를 구하는 문제)

귀류법

범인 A가 어제밤 현장에 없었다. 를 증명하고 싶을 때, -> 명제(결론) 증명

반대로 A가 현장에 있었다고 가정 -> 결론 부정

그런데 A는 밤10시에 서울에서 영화를 보고 있었다고함. -> 모순 발견 -> 결론 부정 틀림!

A가 현장에 없었다. -> 결론이 맞음!

정지문제

프로그램 H : 정지 판별 프로그램 존재

$H(\text{프로그램 } P, \text{ 입력 } X)$

→ $P(x)$ 가 종료하면 H 는 True 반환

→ $P(x)$ 가 무한루프 H 는 False 반환

이 H 를 이용한 새로운 프로그램 D를 만든다.

```
D(P) {  
    if H(P, P) == True  
        while(true)  
            continue  
    else  
        종료  
}
```

정지문제

프로그램 H 는 값이 잘나오는지 판별기이다

프로그램 $P(x)$ 가 값이 잘나오는지 궁금하면

$H(P, x)$ 했을때

값 잘나오면 $\rightarrow$ True

무한루프 $\rightarrow$ False

프로그램 D 는 H에 $P(P)$ 를 물어본 것이다. (프로그램에 프로그램을 넣은것임) (이진수)

자기 자신을 입력으로 판별기에 넣었을때

멈추면 $\rightarrow$ 무한루프

무한루프 $\rightarrow$ 종료(멈추게끔)

정지문제

프로그램 D 는 H에 P(P)를 물어본 것이다.
자기 자신을 입력으로 판별기에 넣었을때

- 멈추면 -> 무한루프
- 무한루프 -> 종료(멈추게끔)

```
D(P) {  
    if H(P, P) == True  
        while(true)  
            continue  
    else  
        종료  
}
```

프로그램 D 에
H에 D(D)를 물어보는 것이다.

```
D(D) {  
    if H(D, D) == True          <- H가 D는 멈춰(정상)  
        while(true)  
            continue          <- 결론 : 무한반복  
    else  
        종료                  <- H가 D 무한반복  
                                <- 결론 : D 멈춤(정상)  
}  
결론 : 모순이다.
```

칸토어의 대각선 논법

자연수와 실수는 같은 개수일까?

자연수 : 1 2 3 4 ...

실수 : 0.15156 0.54546 2.98789 ...

직관적으로는 실수가 많아보임

만약 실수를 전부 나열할 수 있다고 가정 (가정 : 모든 실수를 리스트에 넣었다)

1번 0.123456...

2번 0.234567...

3번 0.568748...

4번 0.235456...

.. 0.1384

5번 0.2495 로 바꾸면 모든 실수와 최소 한자리 이상 다르다. 라는 새로운 숫자 만들기가 가능

결론 : $|R| > |N|$

둘의 관계

정지 문제는 "프로그램도 결국 숫자로 번호 매길 수 있다"는 사실과 칸토어의 대각선 논법을 결합해서 만든 증명이다.

P1

P2

P3

P4

...

모든프로그램의 실행 결과(특히 정지 여부)를 항상 정확하게 판단하는 프로그램은 존재하지 않는다.

마무리

이 책은 타입과 함수, 데이터 구조를 범주론으로 다시 재정리하면서,
코드를 더 추상적이고 일관된 방식으로 설계하는 법을 보여주는 책이다.

수학을 프로그래밍을 위한 수학으로 설명해준다

개별 문법 보다는 구조와 합성 및 여러가지 사상들에 대해 알려준다

초반에는 범주론은 객체와 화살표로 이루어져있고,

범주라는 개념은 프로그래밍 언어 뿐만아니라 여러 구조에서 반복되는 패턴이다.

집합, 작은범주, 순서, 모노이드, Kleisli, product, coproduct, 등 여러 범주 개념 및 규칙 을 알려준다.

프로그램은 값들의 집합이 아니라, 구조를 보존하는 변환들의 세계
이고, 좋은 추상화는 내부보다 관계와 법칙으로 설명된다.

감사합니다